

Legendre transform

It is the Legendre transform that consumes the most computational effort in spherical harmonic formulations. The challenge is to evaluate (for each radial point r_n) a transform from a set of coefficients A_{lm} to values on points θ_j, ϕ_i :

$$A(\theta_j, \phi_i) = \sum_{m=0}^{M-1} e^{im\phi_i} \sum_{l=m}^{L-1} A_{lm} P_l^{|m|}(\cos \theta_j).$$

Performing the sum over l for each j amounts to a matrix multiply (curly brackets) for each m . Then, to evaluate for each i , an FFT is required (square brackets) for each j :

$$\begin{aligned} A(\theta_j, \phi_i) &= \sum_m e^{im\phi_i} \sum_l A_{lm} P_{jlm} = \sum_m e^{im\phi_i} \left\{ \sum_l A_l P_{jl} \right\}_m \\ &= \sum_m e^{im\phi_i} A_{jm} = \left[\sum_m e^{im\phi_i} A_m \right]_j \end{aligned}$$

The whole double sum must be performed for each radial point r_n . For N radial points, and putting $L = M = N$, the transform requires of order N^4 operations.

The most expensive part of the calculation is the the matrix multiply (curly brackets). Parallelisation is implemented by splitting over the indices m and n .

Evaluating the transform and data layout

The spherical harmonic modes present for the case $L = M = 8$ are

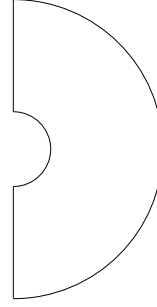
l, m							
0,0	1,1	2,2	3,3	4,4	5,5	6,6	7,7
1,0	2,1	3,2	4,3	5,4	6,5	7,6	
2,0	3,1	4,2	5,3	6,4	7,5		
3,0	4,1	5,2	6,3	7,4			
4,0	5,1	6,2	7,3				
5,0	6,1	7,2					
6,0	7,1						
7,0							

In the code, the modes are rearranged as follows:

l, m			
0,0	1,1	2,2	3,3
1,0	2,1	3,2	4,3
2,0	3,1	4,2	5,3
3,0	4,1	5,2	6,3
4,0	5,1	6,2	7,3
5,0	6,1	7,2	4,4
6,0	7,1	5,5	5,4
7,0	6,6	6,5	6,4
7,7	7,6	7,5	7,4

Modes are then split into, e.g. $N_{th} = 2$ column groups and over $N_r = 3$ cores within each group. ($N_r \times N_{th}$ cores total.)

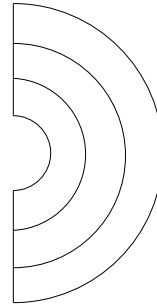
l, m			
	$\vdots$		$\vdots$
0,0	1,1	2,2	3,3
1,0	2,1	3,2	4,3
2,0	3,1	4,2	5,3
3,0	4,1	5,2	6,3
4,0	5,1	6,2	7,3
5,0	6,1	7,2	4,4
6,0	7,1	5,5	5,4
7,0	6,6	6,5	6,4
7,7	7,6	7,5	7,4
$\vdots$		$\vdots$	



Operations that apply independently on each mode are performed with the above split over modes. For each mode, data for all r_n are present.

To evaluate the transform, the data is first split into N_r sections in radius. Within each column group, a transpose is performed between radial points and modes, resulting in all modes for the given column group present on each core:

l, m			
	$\vdots$		$\vdots$
0,0	1,1	2,2	3,3
1,0	2,1	3,2	4,3
2,0	3,1	4,2	5,3
3,0	4,1	5,2	6,3
4,0	5,1	6,2	7,3
5,0	6,1	7,2	4,4
6,0	7,1	5,5	5,4
7,0	6,6	6,5	6,4
7,7	7,6	7,5	7,4
$\vdots$		$\vdots$	

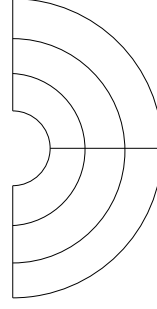


Each transpose involves only N_r cores and requires $N_r - 1$ messages per core. There are N_{th} independent transposes.

Next, sums over l at each θ_j are performed independently on each core. This is the most computationally expensive section of the code, but it is equally balanced across cores.

Note that all l are present for a given m within each column group, and that an equal number of distinct m appear in each group, in this case 4 distinct m .

m	
0	2
7	5
1	3
6	4



After summation of over l , data for all θ_j is present for each m . Within each radial section, data is split over N_{th} sections in θ . A transpose is performed between m -modes and theta points, resulting in data for all m present for each theta section.

Each transpose involves N_{th} cores and requires $N_{th} - 1$ messages per core. There are N_r independent transposes.

The sum over m , i.e. an FFT, is then performed independently within each section. At the end, data for all ϕ_i are present within each r, θ -section.

Overall there are $N_r + N_{th} - 2$ messages per core, each message approximately the same size. Given that the total number of cores is $N_p = N_r \times N_{th}$, the number of messages is minimised for $N_r \approx N_{th}$. The total data sent is approximately independent of the choice of N_r and N_{th} for a given N_p .

In the code

`_Nr` and `_Nth`, from which we get `_Np`, are defined in `parallel.h`. The symbol `_MPI` is only defined if `_Np` is greater than 1.

Two cores have data for the same column group (spectral space) or θ -section (real space) if `rank1/_Nr==rank2/_Nr`.

Two cores have data for the same r -section (physical space) if `modulo(rank1,_Nr)==modulo(rank2,_Nr)`.

`i_H` is the *total* number of harmonics.

`i_H1` is the number of harmonics minus 1 in a *column group*.

The l, m are stored in `var_H%l(nh)` and `var_H%m(nh)`, where `nh=0..i_H1`.

The m present for each group can be found in `var_Th%m(:,group)`, where `group=0.._Nth-1`.